Custom Lab: Determining the Lift Characteristics of an Unknown Airfoil Profile

Jason Khatkar, Jack Meehan, Carlos Zepeda, David LLata, Donald Le, Jose Ramirez Jr, Seong Jae Ahn

Department of Mechanical Engineering, University of California at Berkeley
28 April 2024

Abstract

The purpose of this lab is to use aerodynamic theories to categorize the lift behavior of an unexplored airfoil design. By varying the angle of attack and flow speed in a wind tunnel housing a wing with the desired profile, a range of representative data for the airfoil's coefficient of lift was generated. The wing was manufactured to span the tunnel test section, allowing for an approximation of two-dimensional flow. Lift data was collected along with their respective uncertainty, and the airfoil was found to exhibit symmetrical behavior for positive and negative angles of attack.

Keywords: airfoil, wind tunnel, anemometer, aerodynamics, lift, coefficient of lift

1. Introduction

Our project is driven by an interest in aerodynamics and flight mechanics, particularly in understanding how changes in the angle of attack affect the coefficient of lift. This investigation is not merely academic, but originates from our team's curiosity about the behavior of airfoils under varying conditions and aim to visually validate these effects through empirical measurements in testing. The purpose of this lab is to model the lift coefficient of an unknown airfoil. This is done by varying the angle of attack and wind speed to determine the lift force being applied to the airfoil to then determine the coefficient of lift. We used a wind tunnel to apply various wind velocities onto the airfoil. We are varying the angle of attack through several small angles and discrete wind speeds to model our airfoil and compare that to the aerodynamic theory. Our null hypothesis is that our experiment will adhere to the theoretical values of lift coefficient within the precision error, as well as adhering to the shape we see in other airfoils. With our experiment, we hope to identify the characteristics of the airfoil we are testing. The field of airfoil mechanics is a heavily explored branch of engineering within the aerospace industry. Although our airfoil can't represent the range of Reynold's Numbers that are norm for airplanes in flight, the desire is to capture the coefficient of lift characteristic to a range of Reynold's Numbers so that we can extrapolate it for real world designs. Part of our results should characterize the range of angles of attack that would and would be most effective for generating lift. This would give us the stall angle for the aircraft, which is the most effective singular number for comparison of our airfoil to other known airfoils.

2. Theory

Lift is a force generated by a pressure differential and skin friction effects across a surface in the flow of a fluid. Through lift, the weight of an aircraft is countered and flight can be achieved. Key to achieving lift is the shape of the object, as a flat plate aligned parallel with flow will generate no lift. Hence the development of airfoils, shapes designed to maximize lift and minimize drag. The profile of an airfoil is the key determining factor of the lift generated, and determining the lift characteristics of airfoils is an empirical process. For a given shape of airfoil at a given angle of attack, the equation for lift is given as follows:

$$L = \frac{1}{2} p_\infty V_\infty^2 S C_L \tag{1}$$

Where ρ_∞ is the air density, V_∞ is the velocity, S is the span of the airfoil, and C_L is the coefficient of lift, a dimensionless scaling factor intrinsically tied to the airfoil profile. Unfortunately the lift generated by wings is partially spoiled by induced drag from turbulent vortices spiralling off of the wingtips. For testing purposes it was practical to generalize airfoil data and thus NACA, NASA's

predecessor, developed a methodology for categorizing airfoil lift coefficients. "Infinite wings" are idealized models where the wing is considered to have an infinite span. In this model, with a steady flow approaching the leading edge, the three dimensional aerodynamics can be simplified to two dimensions. By manufacturing airfoils to stretch the length of a wind tunnel test section, the wingtip effects are avoided and general relationships between airfoils and lift coefficients can be developed.

Of course, this relationship is determined both by the flow conditions, which the Reynolds number encompasses, and the angle of attack of the airfoil, which is the airfoil's angle of incline relative to the flow. The Reynolds number is a dimensionless parameter describing flow conditions, and is often used to determine whether a flow is turbulent or laminar. The equation for Reynolds number for an airfoil of chord length c is given below:

$$Re = \frac{\rho_\infty V_\infty c}{\mu_\infty} \tag{2}$$

Here μ_∞ is the dynamic viscosity of the fluid. The angle of attack is measured from the airfoil's chord, the line from the airfoil's tip to its leading edge.

Typically airfoil lift data is reported in a graph of the lift coefficient versus the angle of attack, with each line in the graph being at a distinct Reynolds number. C_L varies linearly with angle for small angles until it eventually peaks in the positive and negative, after which further increases to the angle of attack cause a loss of lift. This condition is known as stalling.

Key to testing airfoil designs is a wind tunnel, which draws air through a contraction cone into a test section, accelerating the air in the process. The basic principle stems from the conservation of mass: the mass flow rate into the wind tunnel equals the mass flow rate out of the wind tunnel.

$$\dot{m}_{in} = \dot{m}_{out} \tag{3}$$

Since the fluid mass flows into a constant area intake and through a constant area test section, assuming steady state conditions, the mass flow rate can be defined as $\ddot{m} = \rho A V$, where A is the cross-sectional area of the flow in question. Plugging this into Eq.3 yields the continuity equation for steady fluid flow:

$$\rho_1 A_1 V_1 = \rho_2 A_2 V_2 \tag{4}$$

For low-speed flow of air, where $V < 100$ m/s, the fluid can be assumed to be incompressible to a close approximation [2]. The maximum airflow in this wind tunnel is approximately 25m/s, thus the assumption of constant density is valid and the term can be removed from Eq.4. Thus, the contraction cone of the windtunnel has the effect of accelerating the air proportional to the ratio of areas.

Importantly, this is only valid with the assumption of uniform flow speed across the entire cross-sectional area. Proceeding with this assumption, we can use a pitot tube to probe the test section and obtain a reading of the total pressure by pointing it directly into the flow. The total pressure p_0 of a fluid in motion is composed of the static pressure p and dynamic pressure q as follows:

$$p_0 = p + q = p + \frac{1}{2}\rho_\infty V_\infty \tag{5}$$

Thus we can obtain values for test section velocity using a pitot tube to measure the total pressure and another sensor to measure the static pressure. Since the flow can be approximated as incompressible, the static pressure is equal to the atmospheric pressure.

Thermal anemometers are also used to make an additional measurement of the free-stream velocity in the test section. These work by monitoring the amount of heat removed from a small, electrically heated wire as the fluid flows over it. For a fluid with known physical properties like air, the heat flux can be solved for the velocity of the fluid causing convection.

3. Methods

This lab utilizes the built-in pressure transducer, pitot tube, and strain gauge built in to the wind tunnel, as well as a separate hot wire anemometer and angle gauge, to measure the wind speed, angle of attack, and lift force produced by the wing. Below is a list of the sensors used:

- Hot Wire Anemometer (Hot Wire Thermo-Anemometer Model 407123): Measures and displays wind speed perpendicular to small opening, with accuracy of $\pm(3.0\%\ \text{rdg} + 0.3\ \text{m/s})$.
- Pitot Tube (Series 160-18 Stainless Steel Pitot Tube): Measures the dynamic pressure of the air flowing through the test section. Used to calibrate the pressure transducer, and assumed to be perfectly accurate for the purpose of calibrating the pressure transducer
- Pressure Transducer (Omega PX164 Pressure Transducer): Changes resistor value based on pressure to give a digital pressure measurement with an accuracy of ± 1.19 V.
- Strain Gauge (custom): Measures position as a change in current of the element as the voltage times over a change in resistance of a coil that is strain. Due to a lack of testing information, it is assumed to be perfectly accurate.
- Angle Gauge (GemRed Digital Angle Gauge): Measures the angle of incline of the flat surface it rests on based on a predefined reference, accurate to 0.05 degrees.

Prior to collecting lift data, both the pressure transducer and the strain gauge were calibrated. The pressure transducer was calibrated using the pitot tube also housed in the wind tunnel. The full range of the wind tunnel's capabilities was explored, varying the variable frequency drive controlling the fan in steps of 100 from a baseline of 0 up to 1700. At each step, many data points were collected. A line was fit mapping the pressure transducer's output voltage to the pitot tube's pressure reading, which was converted from millimeters of mercury to Pascals. The results of this calibration are plotted below in figures 1 and 2:

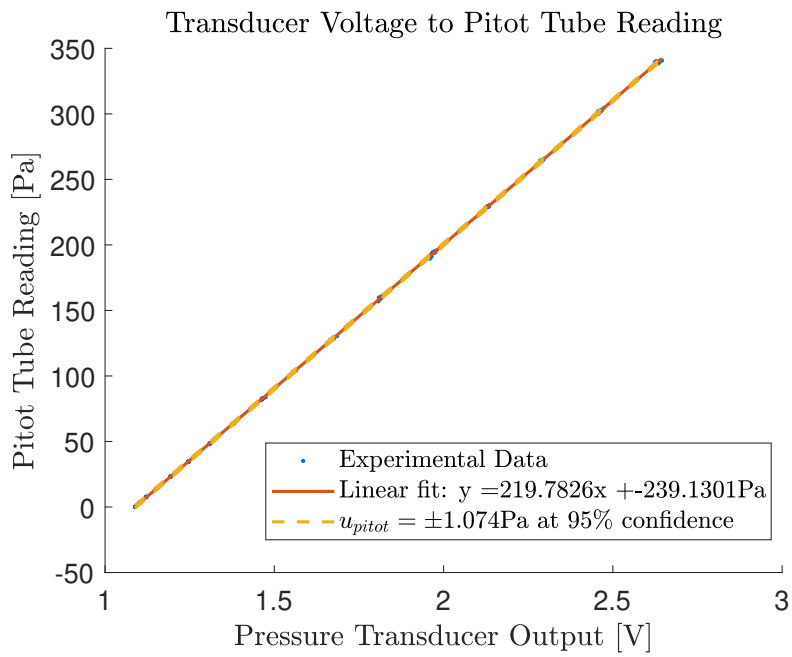


Figure 1: Pitot Tube reading VS Transducer Voltage reading. Pitot Tube reading takes the gauge pressure, transducer reading was taken directly from sensor.

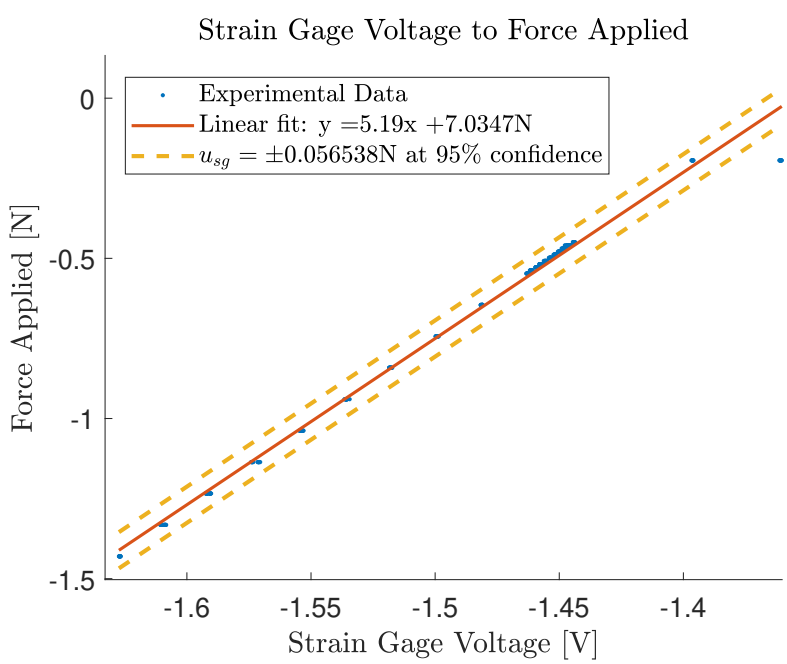


Figure 2: Strain Gage voltage VS Applied Force. Strain readings taken directly from the sensor and force is measured precisely using small weights

By approximating the flow as incompressible, a simple formula is derived to calculate the flow speed from the dynamic pressure. This theory has been proven in literature to be highly accurate for flow at speeds lower than Mach 0.3, and the wind tunnel of this experiment does not exceed

this limit.

Next, the anemometer was used to collect direct measurements of wind speed at each of the same VFD control settings. This data would be used to compare with velocity measurements obtained from the calibrated, converted pressure transducer output.

The strain gauge was calibrated by placing weights of known mass onto a platform, also of known weight, hanging onto the sensor. Weights were added incrementally in steps of 1g from 0g to 10g, and then in steps of 10g up to 100g. The process was then reversed back down to 0g of added weight (not including the hanger assembly) to account for potential hysteresis. Gauge voltage values were then fit to the known force of gravity resulting from the added masses. As shown in the graph below, there is a strong linear relationship between the force applied and the output voltage, though the last data points collected to not fit the curve. This suggests error from hysteresis for low force values.

Following calibration, the airfoil was affixed to the strain gauge and the sensor's output was measured at wing angles of attack in the range $[-16, +16]$ degrees, at VFD settings of 800, 1200, and 1600 for each degree setting. By varying degree in steps of 2, a sufficient amount of data is acquired to determine the general airfoil behavior. The angle gauge was used here in conjunction with a negative printout of the airfoil's profile to ensure it rested on a surface parallel to the chord. The angle gauge was calibrated on the bottom of the test section and then placed on the airfoil, which was then rotated to within ± 0.1 degrees of the desired attitude. Flow speeds and their accompanying Reynolds number across the wing were chosen based on the VFD's output limitations, at a value near the maximum output and two other values below that in multiples of 400. Typical lift coefficient plots have data for Reynolds numbers beginning at 10^6 and increasing thereafter, but we were limited by the constraints of this wind tunnel. The limits of the angle of attack range were determined by review of typical airfoil stalling behavior. Many commercial airfoils begin to stall around 10 to 15 degrees in either direction, so 16 degrees was chosen to encompass stall behavior.

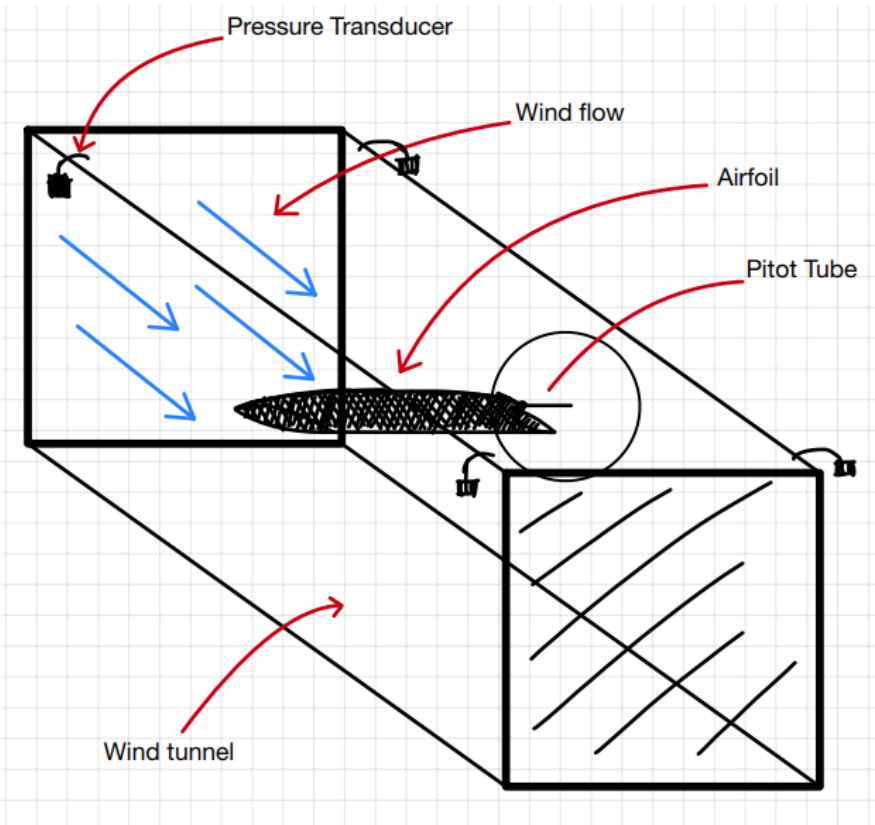


Figure 3: The diagram showcases the wind tunnel setup during experimentation

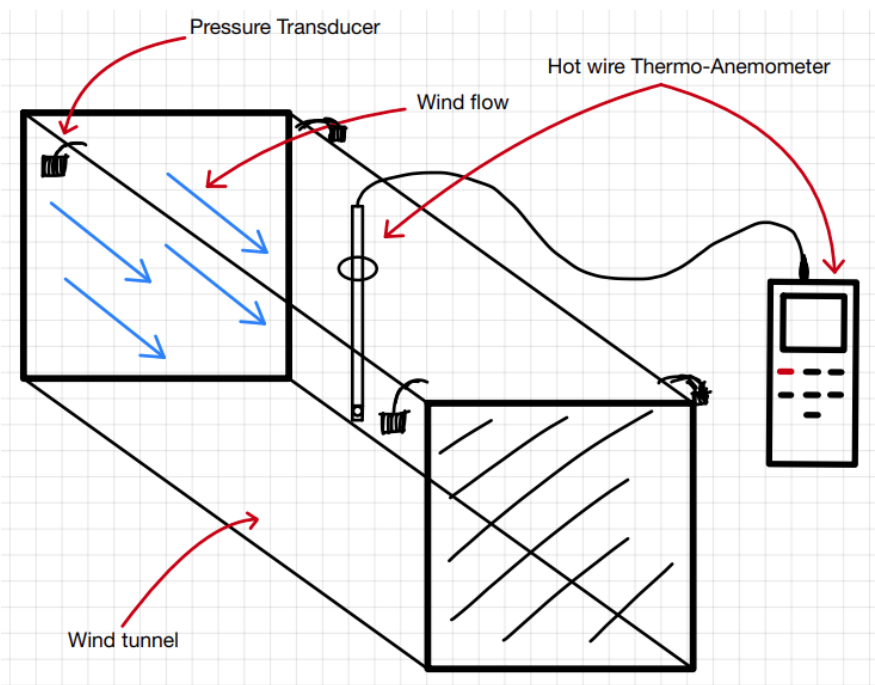


Figure 4: The wind tunnel setup, depicting positions of the anemometer and PT for calibration.

4. Results and Discussion

From our experiment, We were able to successfully characterize the coefficient lift of our custom airfoil across a range of angles of attack. This was done for three separate Reynold's numbers Re=69,000 (figure 8), Re=120,000 (figure 9), and Re=160,000 (figure 10). For our results, the coefficient of lift is calculated using Equation 1 from the theory section. For the lift force strain gauge measurements were used and for wind speed two separate sensors, an anemometer, and a pressure transducer (derived) were utilized. The relationship between the coefficient of lift and angle of attack for both sensors is plotted along with their uncertainties for comparison and verification.

Listed below are various sources of uncertainty that were identified and included or excluded from our results based on the following justifications.

- Strain gauge: 95% confidence interval taken from calibration curve created using known weights
- Pressure transducer: 95% confidence interval taken from calibration curve created using a pitot tube
- Anemometer: $\pm(3.0\%\ \text{rdg} + 0.3\ \text{m/s})$ taken from the user manual [3]
- Angle Gauge: $\pm(0.2°)$ taken from the specification sheet [6]
- Measuring tape: $\pm(1/16")$ from the resolution of the measuring tape

Using the Gaussian prop of uncertainty equation we can calculate the uncertainty of our coefficient of lift know the uncertainty from the various sources.

$$U = \sqrt{(\frac{\partial y}{\partial x_1})^2 + (\frac{\partial y}{\partial x_2})^2 + ... + (\frac{\partial y}{\partial x_n})^2} \tag{6}$$

Uncertainty propagation calculation - anemometer

$$U = \sqrt{(\frac{U_{sg}}{0.5 p_\infty V_\infty^2 S})^2 + (\frac{LU_{an}}{p_\infty V_\infty^3 S})^2 + (\frac{LU_{sa}}{0.5 p_\infty V_\infty S^2})^2} \tag{7}$$

Uncertainty propagation calculation - pressure transducer

$$U = \sqrt{(\frac{U_{sg}}{0.5 p_\infty V_\infty^2 S})^2 + (\frac{LU_{pt}}{p_\infty V_\infty^3 S})^2 + (\frac{LU_{sa}}{0.5 p_\infty V_\infty S^2})^2} \tag{8}$$

- U_{sg} - Strain Gage Uncertainty: Calculated from the calibration curve
- U_{pt} - Pressure Transducer Uncertainty: Calculated from the calibration curve
- U_{an} - Anemometer Uncertainty
- U_{sa} - Surface Area Uncertainty

$$U_{sa} = \sqrt{(wU_m)^2 + (lU_m)^2} \tag{9}$$

- U_m - Measuring Tape Uncertainty

- w - Width
- l - Length

This uncertainty calculation is not holistic and there exist other potential sources of error that were not able to be accounted for which could include:

- User error when taking measurements
- Lack of information/specifications on the strain gauge
- Air viscosity and density values

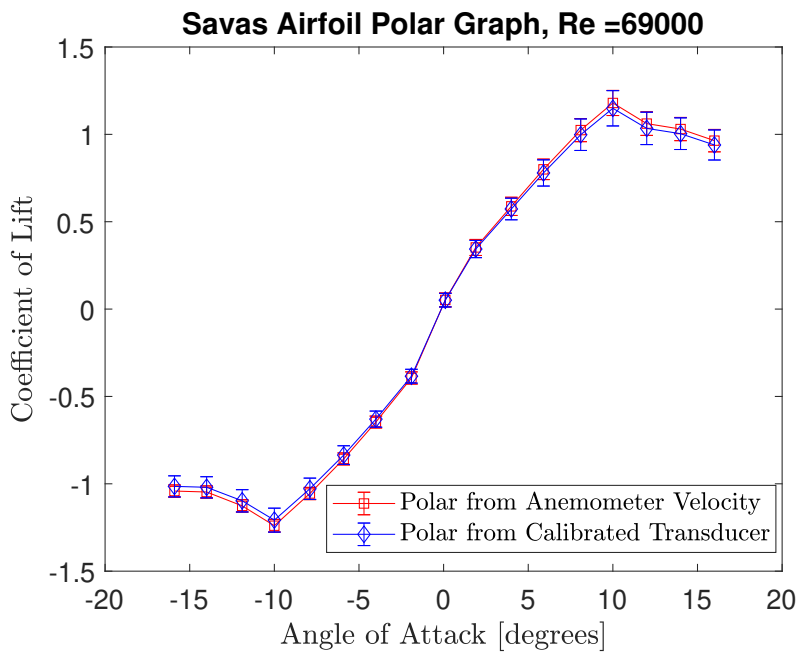


Figure 5: Savas Airfoil Coefficient of Lift graph vs Angle of Attack for Reynold's Number 69,000. C_L is calculated using the Pressure Transducer and Anemometer separately.

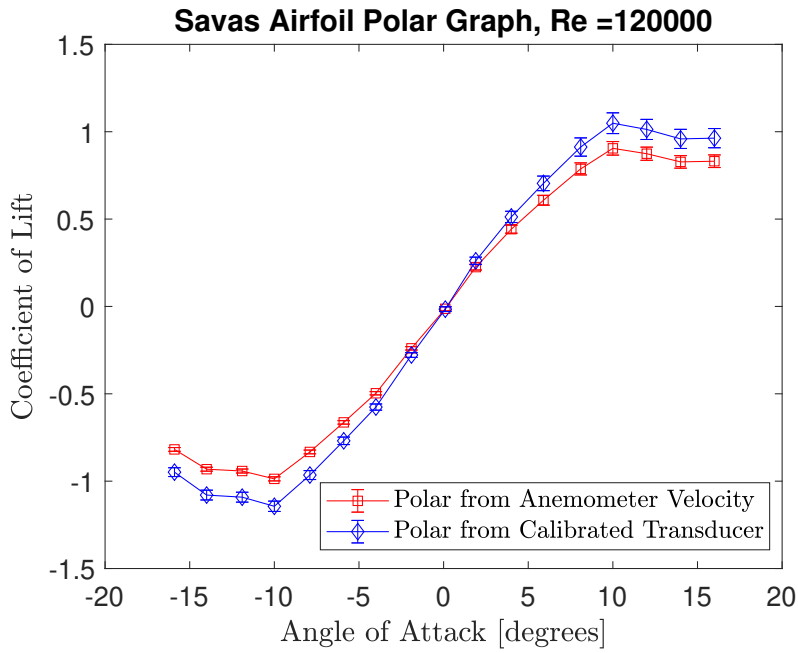


Figure 6: Savas Airfoil Coefficient of Lift graph vs Angle of Attack for Reynold's Number 120,000. C_L is calculated using the Pressure Transducer and Anemometer separately.

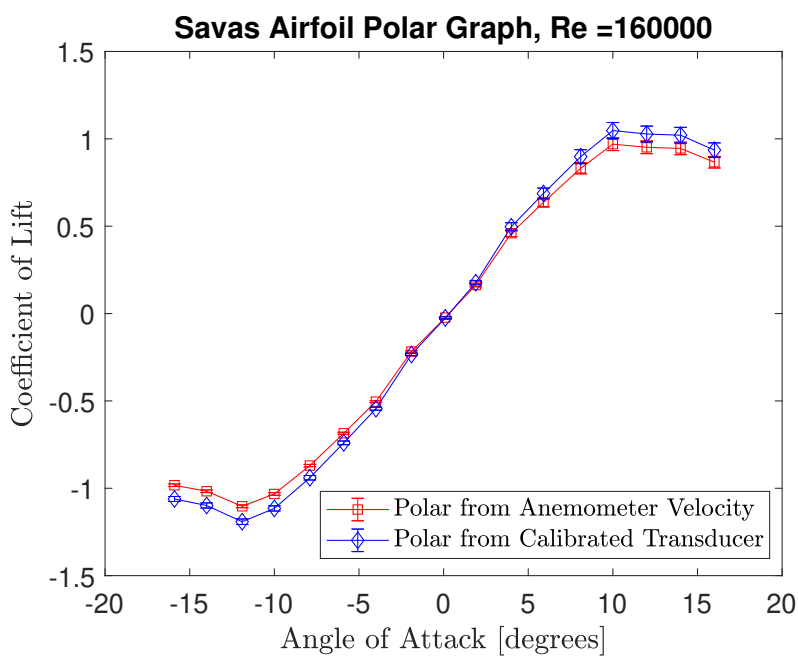


Figure 7: Savas Airfoil Coefficient of Lift graph vs Angle of Attack for Reynold's Number 160,000. C_L is calculated using the Pressure Transducer and Anemometer separately.

From our experiment, we observed a symmetrical lift behavior of the airfoil, consistent with our hypotheses, however, due to the wind tunnel's speed limitations, we could not fully explore the

Table 1: Coefficient of Lift given by Pressure Transducer Data

Angle of Attack	**Re=69000**	**Re=120000**	**Re=160000**
-16°	-1.01 ± 0.06	-0.95 ± 0.03	-1.06 ± 0.01
-14°	-1.02 ± 0.06	-1.08 ± 0.03	-1.10 ± 0.01
-12°	-1.10 ± 0.06	-1.09 ± 0.03	-1.19 ± 0.02
-10°	-1.21 ± 0.07	-1.14 ± 0.03	-1.12 ± 0.01
-8°	-1.03 ± 0.06	-0.97 ± 0.03	-0.94 ± 0.01
-6°	-0.84 ± 0.05	-0.77 ± 0.02	-0.74 ± 0.01
-4°	-0.63 ± 0.05	-0.58 ± 0.02	-0.54 ± 0.01
-2°	-0.38 ± 0.04	-0.28 ± 0.01	-0.23 ± 0.01
0°	0.05 ± 0.04	-0.02 ± 0.01	-0.02 ± 0.01
2°	0.34 ± 0.05	0.26 ± 0.02	0.18 ± 0.01
4°	0.57 ± 0.06	0.51 ± 0.03	0.50 ± 0.02
6°	0.78 ± 0.07	0.70 ± 0.04	0.69 ± 0.03
8°	1.00 ± 0.09	0.91 ± 0.05	0.90 ± 0.04
10°	1.15 ± 0.10	1.05 ± 0.06	1.05 ± 0.05
12°	1.03 ± 0.09	1.01 ± 0.06	1.03 ± 0.05
14°	1.00 ± 0.09	0.96 ± 0.05	1.02 ± 0.04
16°	0.94 ± 0.09	0.96 ± 0.05	0.94 ± 0.04

Table 2: Coefficient of Lift given by Anemometer Data

Angle of Attack	**Re=69000**	**Re=120000**	**Re=160000**
-16°	-1.04 ± 0.04	-0.82 ± 0.01	-0.98 ± 0.01
-14°	-1.05 ± 0.04	-0.93 ± 0.01	-1.02 ± 0.01
-12°	-1.13 ± 0.04	-0.94 ± 0.01	-1.10 ± 0.01
-10°	-1.24 ± 0.04	-0.99 ± 0.01	-1.03 ± 0.01
-8°	-1.06 ± 0.04	-0.83 ± 0.01	-0.87 ± 0.01
-6°	-0.86 ± 0.03	-0.66 ± 0.01	-0.69 ± 0.01
-4°	-0.65 ± 0.03	-0.50 ± 0.01	-0.50 ± 0.01
-2°	-0.39 ± 0.03	-0.24 ± 0.01	-0.22 ± 0.00
0°	0.05 ± 0.04	-0.01 ± 0.01	-0.02 ± 0.01
2°	0.35 ± 0.05	0.23 ± 0.02	0.16 ± 0.01
4°	0.59 ± 0.05	0.44 ± 0.02	0.46 ± 0.02
6°	0.80 ± 0.06	0.61 ± 0.03	0.64 ± 0.03
8°	1.02 ± 0.07	0.79 ± 0.03	0.83 ± 0.03
10°	1.18 ± 0.07	0.90 ± 0.04	0.97 ± 0.04
12°	1.06 ± 0.07	0.87 ± 0.04	0.95 ± 0.04
14°	1.03 ± 0.07	0.83 ± 0.04	0.95 ± 0.04
16°	0.96 ± 0.06	0.83 ± 0.04	0.87 ± 0.03

effects of a higher Reynolds number of lift. Despite theses limitations, our results provide a solid foundation for understanding the airfoil performance which follows our theoretical graphs.

5. Conclusion

The outcomes of our experiment have effectively validated our hypothesis, affirming that the airfoil indeed generates a symmetrical lift polar graph. This comprehensive analysis of lift demonstrates that symmetrical airfoils prioritize attributes such as maneuverability, precise control, and operational simplicity. Consequently, they emerge as optimal choices for applications requiring agile control surfaces, high-speed flight, and dynamic aerobatics. However, our findings also underscore the inherent limitations of symmetrical airfoils. Notably, they may not be the most suitable option in scenarios demanding heightened lift or reduced stalling speeds. In such contexts, cambered airfoils emerge as a more appropriate choice. By clarifying these distinctions, our study augments the nuanced understanding of airfoil dynamics. Overall, our experiment contributes significantly to the advancement of aerodynamic knowledge and informs prudent airfoil selection practices across aviation domains.

Contributions

Jason Khatkar worked on intro, appendix, references, figure display and captioning, methods, formatting and logistics.

Seongjae Ahn partly worked on intro, methods, discussion.

Jack Meehan developed the code for sensor calibration, lift coefficient calculation and plotting, and the uncertainty calculations. He also worked on theory and methods.

Carlos Zepeda worked on the theory section.

Jose Ramirez Jr drew diagrams and worked on the introduction and conclusion sections.

Donald Le worked on the results and discussion section.

References

[1] White, F. M., *Fluid Mechanics*, 4th ed. Blacklick, OH, USA: McGraw-Hill, 1998. Accessed on April 18, 2024

[2] J. D. Anderson, *Introduction to Flight*, 8th ed. New York, NY: McGraw-Hill, 2015. Accessed on April 18, 2024

[3] Manufacturer, *User's Manual: Hot Wire Thermo-Anemometer Model 407123*, Version 6.6, EXTECH Instruments, 2016. Accessed on March 14, 2024

[4] R. Eppler, *Airfoil Design and Data*, Springer, Berlin, Heidelberg, 1990. Accessed on April 16, 2024

[5] Manufacturer's Name, *Series 160 Stainless Steel Pitot Tube Specification Sheet*. [Online]. Available: `https://dwyer-inst.com/en/series-160-stainless-steel-pitot-tube.html`. [Accessed: April 25, 2024].

[6] GemRed. *GemRed Digital Angle Gauge (Green Backlight) Specification Sheet*. Manufacturer's Specification Sheet. Year of Publication. URL: `https://www.gemred.com/product/gemred-digital-angle-gauge-precise-handy/` [Accessed: April 25, 2024].

[7] Omega. *Omega PX164 Pressure Transducers Specification Sheet*. Manufacturer's Specification Sheet. Year of Publication. URL: `https://assets.omega.com/manuals/test-and-measurement-equipment/pressure/pressure-transducers/M0260.pdf` [Accessed: April 25, 2024].

Appendix

Appendix A: Plots and Figures

Appendix B: MATLAB/other codes

Table of Contents

Strain Gage Calibration 1

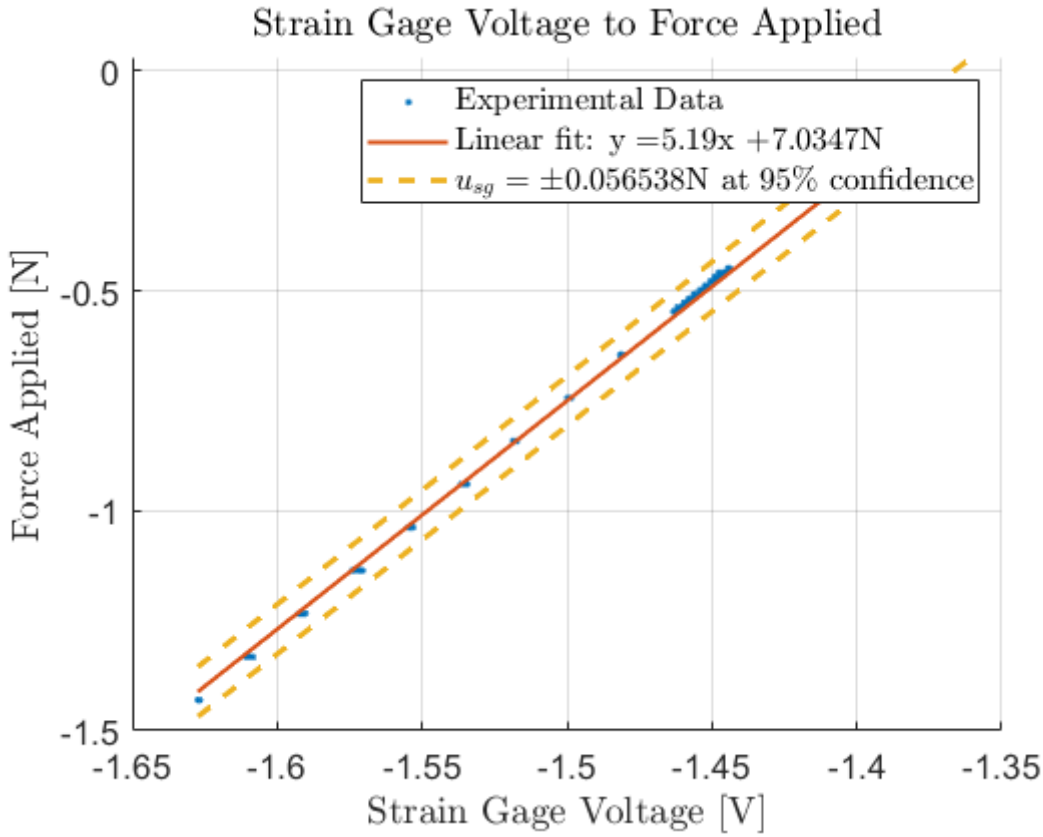
Strain Gage Calibration

This code compares the weight placed on the strain gage to the measured sensor response, fitting a curve so that we can obtain lift values.

```
clear all; close all; clc;

% Assumptions
g = 9.81;       % [m/s^2]

fileName = 'processed strain cal.xlsx';
table = readtable(fileName);
sgV = table.StrainGage_V_;
weight = table.Mass .* (1/1000) .* -9.81; % convert g to kg and kg to N

[pfit_sg,S_sg] = polyfit(sgV,weight,1);
[~,delta] = polyval(pfit_sg,sgV,S_sg);
unc_fit_sg = delta(1)*2; % 95% CI of fit

x1 = min(sgV);
x2 = max(sgV);
y1 = polyval(pfit_sg,x1);
y2 = polyval(pfit_sg,x2);

f1 = figure(1);
hold on
p1 = scatter(sgV,weight,'.','DisplayName','Experimental Data','LineWidth',3);
p2 = plot([x1,x2],[y1,y2],'LineWidth',1.5,'DisplayName',...
    strcat('Linear fit: y = ',num2str(pfit_sg(1)),'x + ',...
    num2str(pfit_sg(2)),'N'));
p3 = plot([x1,x2],[y1+unc_fit_sg,y2+unc_fit_sg],...
    'LineWidth',2,'LineStyle','--','DisplayName',...
    strcat('$u_{sg} = \pm$',num2str(unc_fit_sg),'N at 95$\%$ confidence'));
plot([x1,x2],[y1-unc_fit_sg,y2-unc_fit_sg],...
    'LineWidth',2,'LineStyle','--','Color',[0.9290, 0.6940, 0.1250]);
grid on
xlabel('Strain Gage Voltage [V]','Interpreter','Latex');
ylabel('Force Applied [N]','Interpreter','Latex');
title('Strain Gage Voltage to Force Applied','Interpreter','Latex');
legend([p1,p2,p3],'Interpreter','Latex');
fontsize(f1,scale=1.3);


save('straingage_fit','pfit_sg','unc_fit_sg');
```

Warning: Column headers from the file were modified to make them valid MATLAB identifiers before creating variable names for the table. The original column

```
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
```



Published with MATLAB® R2023b

Table of Contents

Measured Velocity Data Processing 1
Getting Data 1
Pitot Tube -> Transducer Calibration 3
Anemometer -> Transducer Calibration 5

Measured Velocity Data Processing

This code takes the pitot tube pressure, contraction cone pressure, and manually-entered anemometer data, processing it for usage in lift calculations.

```
clear all; close all; clc;


% Assumptions
pstatic = 1.013*10^5; % Atmospheric pressure [Pa]
rho = 1.225; % Air density at STP [kg/m^3]
c = 133.3; % conversion factor [Pa/mmHg]
```

Getting Data

```
fileName = 'processed vel data.xlsx';
sheetNames = sheetnames(fileName);
tables = cell(length(sheetNames), 1);
pressTrans = [];
PTstd = [];
pitot = [];
for i = 1:length(sheetNames)
    % Read table from the current sheet
    tables{i} = readtable(fileName, 'Sheet', sheetNames{i});
    pressTrans = [pressTrans;tables{i}.Press_Trans__V_];
    pitot = [pitot;tables{i}.PitotTube];
end

pitot = pitot.*c; % conversion from mmHg to Pa
```

```
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
```

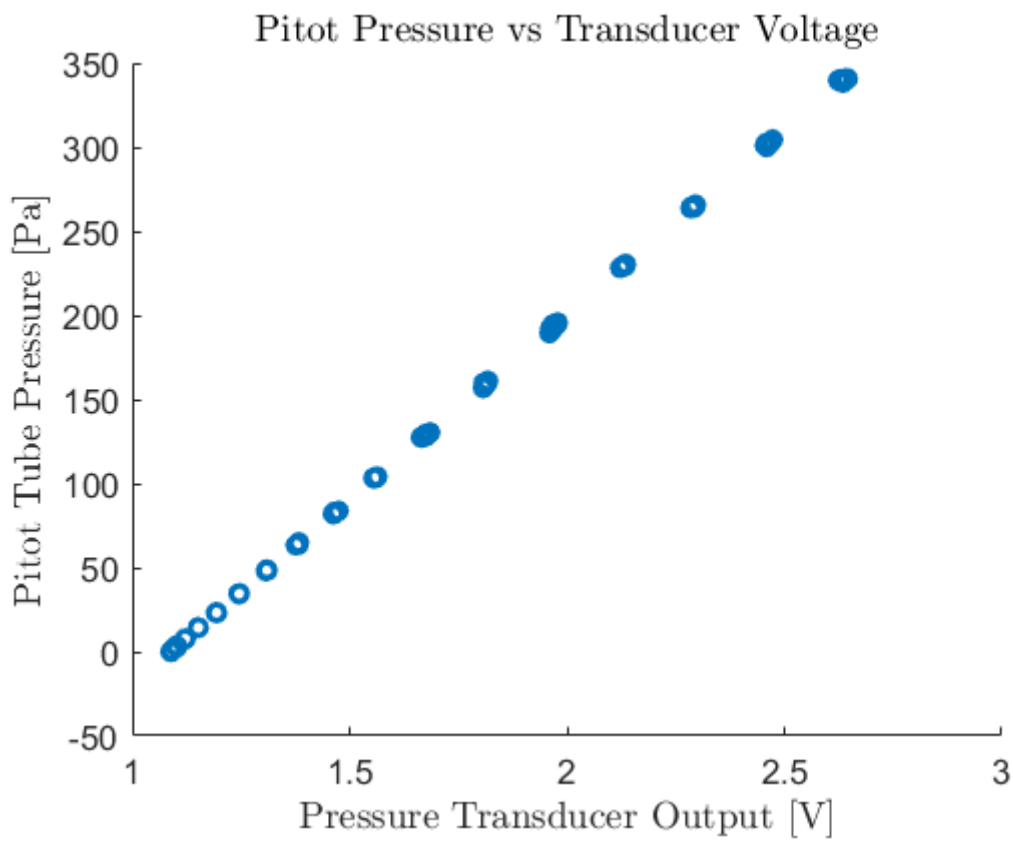
```
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
```

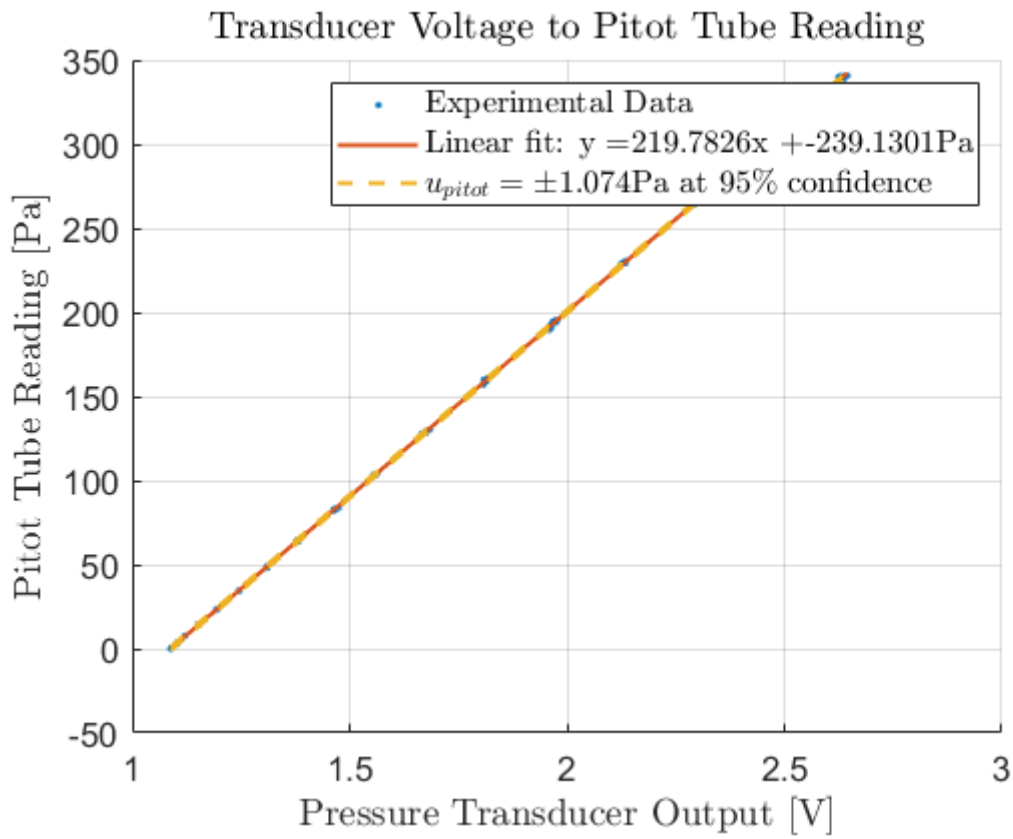
```
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
```

Pitot Tube -> Transducer Calibration

```
f1 = figure(1);
scatter(pressTrans,pitot,"LineWidth",2)
xlabel("Pressure Transducer Output [V]",'Interpreter','Latex');
ylabel("Pitot Tube Pressure [Pa]",'Interpreter','Latex');
title("Pitot Pressure vs Transducer Voltage",'Interpreter','Latex')
fontsize(gcf,scale=1.3)

p_total = pitot + pstatic; % total pressure = static + dynamic
vel_pitot = abs(sqrt(pitot*2/rho)); % above eqn solved for velocity

[pfit_pt,S_pt] = polyfit(pressTrans,pitot,1);
[~,delta] = polyval(pfit_pt,pressTrans,S_pt);
unc_fit_pt = delta(1)*2; % 95% CI of fit

x1 = min(pressTrans);
x2 = max(pressTrans);
y1 = polyval(pfit_pt,x1);
y2 = polyval(pfit_pt,x2);

f2 = figure(2);
hold on
p1 = scatter(pressTrans,pitot,'.','DisplayName','Experimental Data');
p2 = plot([x1,x2],[y1,y2],'LineWidth',1.5,'DisplayName',...
    strcat('Linear fit: y = ',num2str(pfit_pt(1)),'x + ',...
    num2str(pfit_pt(2)),'Pa'));
p3 = plot([x1,x2],[y1+unc_fit_pt,y2+unc_fit_pt],...
    'LineWidth',1.5,'LineStyle','--','DisplayName',...
```

```
    strcat('$u_{pitot} = \pm$',num2str(unc_fit_pt),'Pa at 95$\%$
confidence'));
plot([x1,x2],[y1-unc_fit_pt,y2-unc_fit_pt],...
    'LineWidth',1.5,'LineStyle','--','Color',[0.9290, 0.6940, 0.1250])
grid on
xlabel('Pressure Transducer Output [V]','Interpreter','Latex')
ylabel('Pitot Tube Reading [Pa]','Interpreter','Latex')
title('Transducer Voltage to Pitot Tube Reading','Interpreter','Latex')
legend([p1,p2,p3],'Interpreter','Latex')
fontsize(gcf,scale=1.3)
```





Anemometer -> Transducer Calibration

```
anemometer = [0,0.8,2.4,3.7,5.1,6.3,7.6,9.3,10.5,11.7,...
                13.2,15.1,17.0,19.0,20.2,22.1,23.4,25.7]; % [m/s]
n = [10,13,10,10,11,10,10,10,10,10,10,10,10,10,10,9,10,10];
anemometer = repelem(anemometer,n); % make nice array of same size
                                    % that corresponds to pressTrans

[pfit_an,S_an] = polyfit(pressTrans,anemometer,3);
[~,delta] = polyval(pfit_an,pressTrans,S_an);
unc_fit_an = delta(1)*2;

transducerX = linspace(min(pressTrans), max(pressTrans), 50);
fittedAnemY = polyval(pfit_an, transducerX);
err = unc_fit_an*ones(size(transducerX));

% f3 = figure(3);
% p1 = scatter(pressTrans,anemometer,...
%     "LineWidth",2,'DisplayName','Experimental data');
% hold on;
% p2 = errorbar(transducerX,fittedAnemY,err,...
%     'r-','DisplayName','Fitted curve ($n=3$) with $95\%$ confidence
interval');
% grid on;
% xlabel("Pressure Transducer Output [V]",'Interpreter','Latex');
```

```
% ylabel("Anemometer Output [m/s]",'Interpreter','Latex');
% title("Anemometer Wind Speed vs Transducer Voltage",'Interpreter','Latex');
% legend([p1,p2],'Interpreter','Latex');
% fontsize(gcf,scale=1.3);

save('transducer_fit','pfit_pt','unc_fit_pt','pfit_an','unc_fit_an')
```

Published with MATLAB® R2023b

Table of Contents

Lift Calculation and Lift Polar Plotting 1
Re Setup 1
Load Data 1
Calculate offset due to weight of extension and airfoil 1
Welcome to the Machine 2

Lift Calculation and Lift Polar Plotting

This code does the heavy lifting, calculating the lift for the range of AoA and Re values, as well as the uncertainty in those values. Lift polar graphs are then plotted, 3 curves for 3 Re numbers / velocity values.

```
clear all; close all; clc;
load('transducer_fit.mat'); load('straingage_fit.mat');
```

Re Setup

Note: Airfoil lift polar usually reported for a range of Re_L values, usiing the length L of the airfoil's chord

```
nu = 1.5111*10^-5; % air kinematic viscosity at 20degC [m^2/s]
rho = 1.225; % Air density at STP [kg/m^3]
L = (4 + 1/2) / 39.37; % chord length from inch to meters [m]
W = (9 + 7/8) / 39.37; % width of airfoil [m]
unc_LW = (1/16) / 39.37; % uncertainty in L and W is the resolution of the
                         % tape measure, 1/16"
S = L*W;
unc_S = sqrt((W*unc_LW)^2+(L*unc_LW)^2);
```

Load Data

```
fileName = 'processed lift3vels.xlsx';
vfd0 = readtable(fileName, 'Sheet', '0vfd');
vfd800 = readtable(fileName, 'Sheet', '800vfd');
vfd1200 = readtable(fileName, 'Sheet', '1200vfd');
vfd1600 = readtable(fileName, 'Sheet', '1600vfd');
```

Calculate offset due to weight of extension and airfoil

```
offsetSGV = vfd0.StrainGage_V_;
uncOffSGV = 1.96*std(offsetSGV);
offsetSGV = mean(offsetSGV);
offsetSGV = -1*pfit_sg(2)/pfit_sg(1) - offsetSGV;
```

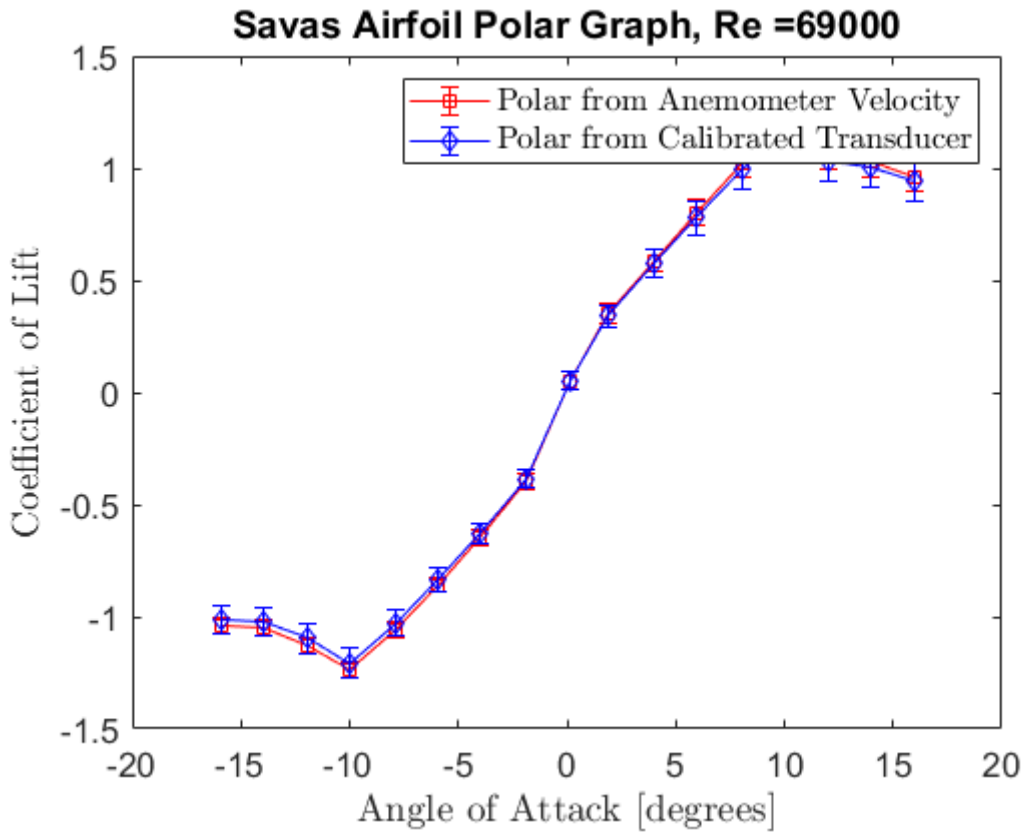
Warning: Column headers from the file were modified to make them valid MATLAB identifiers before creating variable names for the table. The original column headers are saved in the VariableDescriptions property.

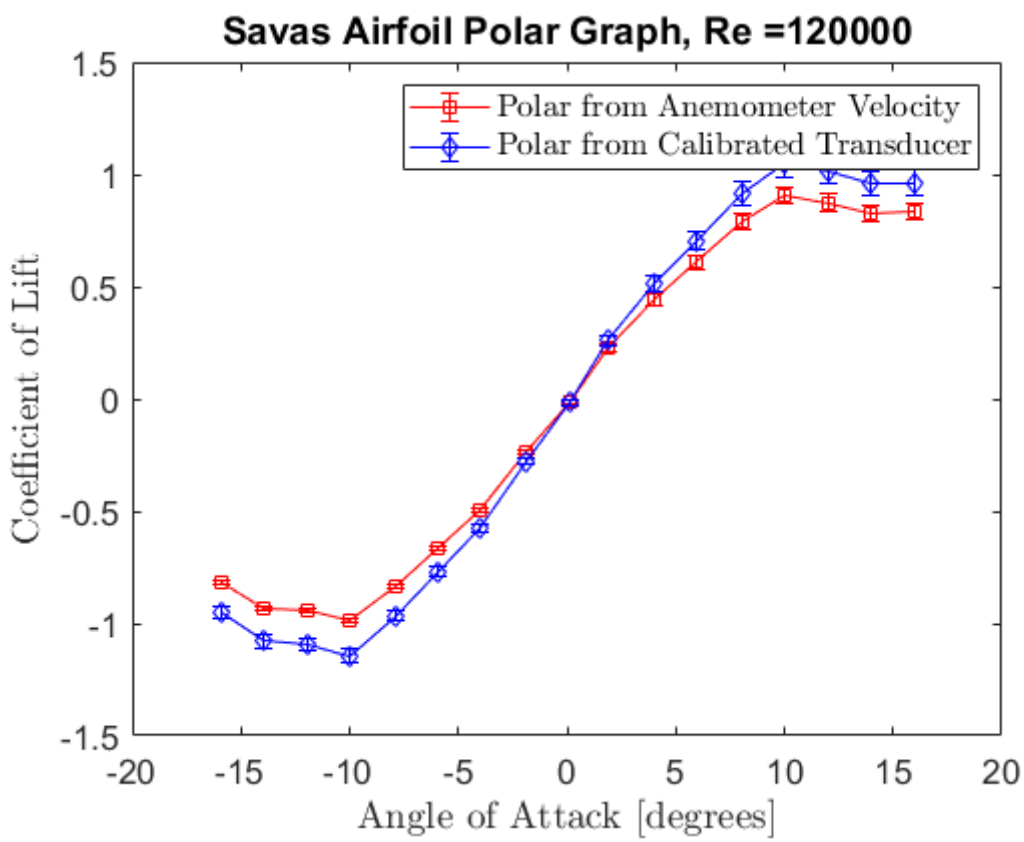
```
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
Warning: Column headers from the file were modified to make them valid MATLAB
identifiers before creating variable names for the table. The original column
headers are saved in the VariableDescriptions property.
Set 'VariableNamingRule' to 'preserve' to use the original column headers as
table variable names.
```

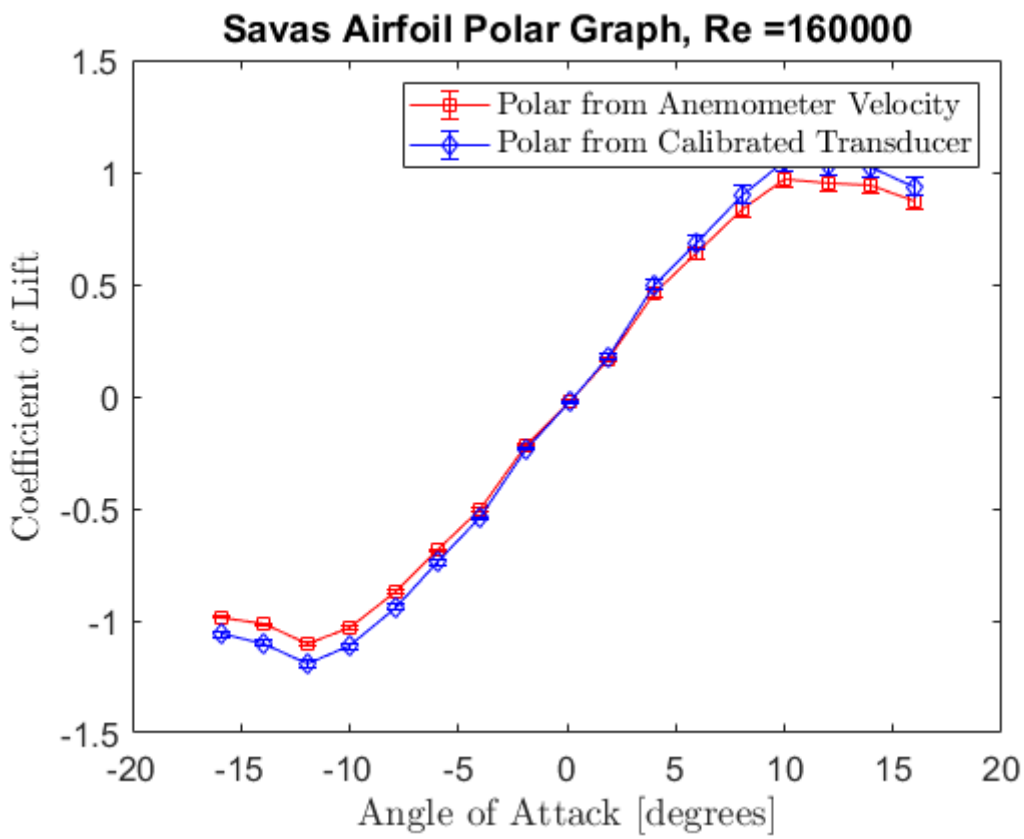
Welcome to the Machine

```
tbs = {vfd800,vfd1200,vfd1600};
C_L_an = [];
C_L_pt = [];
vel_an = [];
vel_pt = [];
unc_L_an = [];
unc_L_pt = [];
% uncertainty in degree measurement, not plotted on graph bc tiny
unc_deg = 0.05*ones(17,1);
% gaussian using uncertainty in fit and uncertainty from device
unc_pt =
sqrt(unc_fit_pt^2+sqrt((0.01*5*pfit_sg(1))^2+(0.0025*5*pfit_sg(1))^2));
Re = [];

% for each wind speed, generates polar graph and calcs uncertainty
for i = 1:length(tbs)
    % subtracts the weight offset in the reading due to airfoil and extens
    tbs{i}.StrainGage_V_ = tbs{i}.StrainGage_V_ + offsetSGV;
    uniqueDegrees = unique(tbs{i}.Degree);
    sg_means = zeros(length(uniqueDegrees), 1);
    sg_stds = zeros(length(uniqueDegrees), 1);
    % this loop groups rows with same AoA and finds mean & STD lift force
    for j = 1:length(uniqueDegrees)
        currentDegree = uniqueDegrees(j);
        rowsForDegree = tbs{i}.Degree == currentDegree;  % Logical indexing
        sgForDegree = tbs{i}.StrainGage_V_(rowsForDegree);

        sg_means(j) = mean(sgForDegree);
        sg_stds(j) = std(sgForDegree);
    end
    % average vel for a run, assumed constant and any variance is due to
    % random fluctuation, not because of actual flow changes due to AoA
    ptV = mean(tbs{i}.PressTrans_V_);
```

```
    ptV_STD = std(tbs{i}.PressTrans_V_);
    % uses anemometer fit to find airspeed
    vel_an = [vel_an,(ptV^3)*pfit_an(1) + (ptV)^2*pfit_an(2) + ...
        ptV*pfit_an(3) + pfit_an(4)];
    % from anemometer spec sheet
    unc_an = vel_an(i)*0.03 + 0.3;
    % calibrates transducer reading, from Volts to Pascals
    pt_in_Pa = ptV*pfit_pt(1) + pfit_pt(2);
    % using fluid mechanics theory, dynamic pressure solved for velocity
    vel_pt = [vel_pt,abs(sqrt(pt_in_Pa*2/rho))];
    % calibrates strain gage reading, Volts to Newtons
    lift = sg_means.*pfit_sg(1) + pfit_sg(2);
    % gaussian prop of uncertainty, have to do individual partial deriv's
    unc_L_an = [unc_L_an,sqrt((unc_fit_sg/(0.5*rho*S*vel_an(i)^2))^2+ ...
        (lift*unc_an/(rho*vel_an(i)^3*S)).^2) + (unc_S.*lift./
(0.5*rho*vel_an(i)^2*S^2))];
    unc_L_pt = [unc_L_pt,sqrt((unc_fit_sg/(0.5*rho*S*vel_pt(i)^2))^2+ ...
        (lift*unc_pt/(rho*vel_pt(i)^3*S)).^2) + (unc_S.*lift./
(0.5*rho*vel_pt(i)^2*S^2))];
     % calculates and saves coefficient of lift
    C_L_an = [C_L_an,lift./(0.5*rho*S*vel_an(i)^2)];
    C_L_pt = [C_L_pt,lift./(0.5*rho*S*vel_pt(i)^2)];
    % Re number not necessary, but usually reported with polar graphs
    Re = [Re,0.5*(vel_an(i)+vel_pt(i))*L/nu];
    figure(i)
    p1 = errorbar(uniqueDegrees,C_L_an(:,i),unc_L_an(:,i),unc_L_an(:,i),...
        '-squarer','DisplayName','Polar from Anemometer Velocity');
    hold on
    p2 = errorbar(uniqueDegrees,C_L_pt(:,i),unc_L_pt(:,i),unc_L_pt(:,i),...
        '-diamondb','DisplayName','Polar from Calibrated Transducer');
    xlabel('Angle of Attack [degrees]','Interpreter','Latex');
    ylabel('Coefficient of Lift','Interpreter','Latex');
    title(strcat('Savas Airfoil Polar Graph, Re =
',num2str(round(Re(i),2,"significant"))));
    legend([p1,p2],'Interpreter','Latex');
    fontsize(gcf,scale=1.3);
end
```


Savas Airfoil Polar Graph, Re =69000
Polar from Anemometer Velocity
Polar from Calibrated Transducer
Coefficient of Lift
Angle of Attack [degrees]
1.5
1
0.5
0
-0.5
-1
-1.5
-20
-15
-10
-5
0
5
10
15
20


Savas Airfoil Polar Graph, Re =120000
Polar from Anemometer Velocity
Polar from Calibrated Transducer
Coefficient of Lift
Angle of Attack [degrees]
1.5
1
0.5
0
-0.5
-1
-1.5
-20
-15
-10
-5
0
5
10
15
20



Published with MATLAB® R2023b